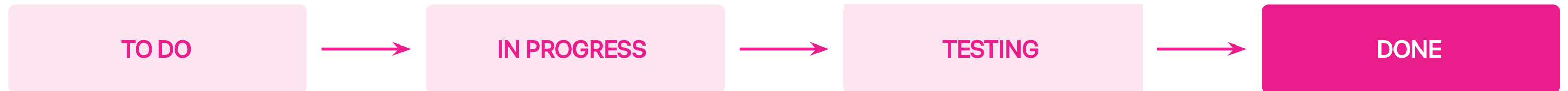


Cómo construir un buen Trello en proyectos frontend

¿Qué es Trello?

Trello es una herramienta visual basada en **Kanban** (una palabra japonesa que significa "tablero").

Cada tarea es una tarjeta que se mueve por columnas.



Un Trello debe responder...

- 1 ¿Qué se está construyendo? — Épicas y tarjetas con contexto
- 2 ¿En qué estado está? — Columnas: To Do, In Progress, Done
- 3 ¿Cómo se estructura? — Trazabilidad entre tarjeta, rama y commits

El tablero es el primer sitio donde alguien nuevo mira para entender el proyecto.

3 niveles de organización

Épicas

Grandes bloques de trabajo por área



User stories

Como [rol], quiero [acción] para que [valor]



Tareas técnicas

Pasos concretos del developer

Desgranar el proyecto



Ir paso a paso. Solo se trabaja cuando está todo desgranado.

Ejemplo práctico: Tablero IT Alumni

Simulación de producto real con usuarios. El tablero **se enfoca primero en el usuario.**

Ejemplo: Tarjeta [LOGIC] Implementar cerca i filtros

> Épica: "Vista bolsa de treball i filtros"

> User Story: "Com a exalumne vull visualitzar les ofertes..."

Definition of Done: checklist completado + rama documentada

Èpiques 4

Targeta 1: [US-01] Vista home i navegació general

Targeta 2: [US-02] Vista Registre / Login

Targeta 3: [US-03] vista xarxa d'exalumnes i filtres

Targeta 4: [US-04] Vista borsa de treball i filtres

+ Añade una tarjeta

To do (Tasques tècniques) 0

+ Añade una tarjeta

In progress 0

+ Añade una tarjeta

Testing / Peer review 0

+ Añade una tarjeta

Done 21

[SETUP] Configurar GitFlow (crear branca develop), repositori a GitHub i estructura de carpetes

[SETUP] Inicialitzar projecte amb Vite + TypeScript i netejar codi d'exemple

[SETUP] Definir variables de CSS4 globals (Design System: colors fúcsia, tipografies, espaiats)

[UI] Maquetar vista crea un compte (Formulari estàtic)

[SETUP] Crear l'esquelet del Router bàsic (SPA) per poder canviar de pestanya

+ Añade una tarjeta

Ejemplo práctico: Tablero API Consumer App

El tablero parte de **bloques técnicos** con ramas Git asociadas.

- **SETUP**

Ej: main, develop, refactor

- **HTML**

Ej: feature/html-structure

- **CSS**

Ej: feature/styles

- **JAVASCRIPT**

Ej: feature/js-setup, feature/js-render, feature/js-fetch, feature/js-axios, feature/js-main

Èpiques4 ...

✓ ÈPICA 1 — SETUP I REFACTOR

✓ ÈPICA 2 — HTML

✓ ÈPICA 3 — CSS

✓ ÈPICA 4 — JAVASCRIPT (FUNCIONALITAT)

+ Añade una tarjeta

To do0 ...

+ Añade una tarjeta

In progress0 ...

+ Añade una tarjeta

Testing0 ...

+ Añade una tarjeta

Done11 ...

render.js

3

✓ [JS] Implementar displayResults() i setupPagination() a render.js

1

✓ [JS] Implementar fetchDataWithFetch() a fetchService.js

✓ [JS] Implementar fetchDataWithAxios() a axiosService.js

✓ [JS] Implementar main.js

1

✓ [REFACTOR]

1

✓ [MERGE] PR final develop →

+ Añade una tarjeta

+ Añade otra lista

<https://trello.com/invite/b/6a33b33da7a850b47b1cf4af/ATTIcd89a3e3d13e92a971a7115d3dcca7ea0770BC81/api-consumer-app>

Buenas prácticas

1. Una tarjeta = una funcionalidad concreta
2. Usar prefijos para identificar el tipo de tarea
3. Checklists obligatorios con pasos técnicos
4. No cerrar tarjetas sin PR mergeada
5. Desglosar bien las tarjetas (error típico: no desglosar el refactor)
6. Método MoSCoW para decidir prioridades

Este flujo se repite para cada tarjeta. Sin excepciones.

Cuando Trello, Git y GitHub están bien conectados, todo queda documentado en el propio flujo de trabajo
